

In a single pass, most local 4-bit executors tested transcribe a detailed plan; two of thirteen bind it

Abstract

Running an established genomic workflow on new input files is execution work: the analytical decisions are already made, and an operator connects the workflow to this batch’s files, runs it, and handles whatever fails. We asked how much detail a separately written plan must supply before a language model can do that execution, using per-sample mitochondrial variant calling as the test. Twelve open-weight models, compressed to 4-bit precision so that they run on a single local machine, and three commercial API models each received one prompt and returned one shell script, with no feedback loop. Without a plan, 6 of 108 local runs and 13 of 27 API runs produced the correct call set; the API models succeeded with a short plan containing no commands, whereas the local models reached 34 of 36 correct runs only when the plan supplied literal command lines. The two model groups also differ in how the models are compressed, served, and sampled, and in size and vendor, so this contrast describes the groups rather than isolating any one cause. Within the plans, a command the model could paste and run raised success where the same command in a non-runnable form did not, and removing the explanatory prose changed nothing we could detect. To separate copying a plan from binding it — connecting its commands to the inputs actually on disk — we moved the reference genome to a new path and stated that path in the input listing: eleven of thirteen local models copied the plan’s stale path and failed, and two used the stated path and succeeded. Shown the failed script and its error message, with up to three attempts, three of the eleven copying models repaired the path in at least two of three seeded repetitions; eight never did. An RNA-seq case study on a public Galaxy server produced counts matching the published benchmark, but a human prepared part of the input and no model completed the final reporting step. These results apply only to workflows whose plans can specify every analytical choice.

Introduction

Routine sequencing analysis requires an operator to connect a fixed workflow to new files, execute it, detect failures, and repeat the steps that failed. We asked whether a language model can take over this execution work from a separately authored plan, and measured it on two benchmarks: per-sample mitochondrial variant calling from paired-end Illumina reads (benchmark 1) and per-sample RNA-seq quantification on a public Galaxy server (benchmark 2). We tested fixed sets of models on one local machine, an NVIDIA Jetson AGX Orin, and through one commercial API. The study did not test analysis design and did not measure human time. Neither benchmark achieved unsupervised operation.

A stable pipeline does not need a language model. A validated shell script, a workflow rule set, a Common Workflow Language (CWL) description, or a saved Galaxy workflow is a better choice because it is deterministic, auditable, and runs without a GPU (Di Tommaso et al. 2017; Mölder et al. 2021). A language model can be useful in the situations a saved pipeline does not cover: a one-time analysis, a change in file layout or tool version, or a natural-language interface for a user who does not write code. It can also be useful for binding—connecting a fixed workflow to a specific set of local inputs. Benchmark 2 tested some parts of binding (Results, *Case study*). A second test moved one reference path, described the move to the model, and thereby examined path binding directly: two of thirteen local models changed the path correctly (Results, *Moving one path separates transcription from binding*).

The separation we tested is the one workflow systems already embody. A specification is written once, and an engine connects it to concrete inputs at run time (Di Tommaso et al. 2017; Mölder et al. 2021; Amstutz et al. 2016; Galaxy Community 2024). Language-model research has produced several methods built on the same split, including plan-and-solve prompting, least-to-most decomposition, ReAct, HuggingGPT, graph dispatch, and Reflexion (Wang et al. 2023; Zhou et al. 2023; Yao et al. 2023; Shen et al. 2023; Kim et al. 2024; Shinn et al. 2023). Alam and Roy (2025) reviewed their application to bioinformatics workflows. Our goal was to measure this design on genomic workflows, with plan detail as the independent variable in benchmark 1.

In practice, the execution side is supplied by an agent harness—software that feeds the model its instructions, runs what the model writes, and returns the output for another round. Several harnesses were in wide use by

mid-2026. We compared none of them, and nothing in our results is known to be harness-independent. For benchmark 2 we used `galaxyproject/loom`, for deployment reasons given in the Supplement. Benchmark 1 has no agent harness at all: the model receives one prompt and returns one script, which we execute externally, with no loop, no observation, and no repair. This design isolates plan detail as the only variable, but it means the benchmark 1 system should not be called agentic. Benchmark 2 was added to supply the missing loop. The closest bioinformatics work evaluates open-ended tool-using research agents against task suites (Huang et al. 2025; Mitchener et al. 2025; Su et al. 2025; Mehandru et al. 2025). We ran none of those suites, so we obtained no external anchor and no number in this paper sits on a published scale. Benchmark 2 has two external comparators, both used with their protocols stated. First, a human executed the same workflow manually on the same server. Second, eight frontier models acting as both planner and executor on the same data cost \$2.82–\$131.83 per run (Nekrutenko 2026). That source is a post by the present author, with unpublished plans and no replicates; it bounds API cost and is not a frontier control.

Whether an open-weight model fits on a given machine depends on two properties that parameter count alone does not capture. A mixture-of-experts (MoE) model routes each token to a subset of its experts, so its per-token compute is governed primarily by the active parameters, whereas weight residency—the memory the loaded weights occupy—is governed by the total parameters. Key-value-cache memory grows linearly with context length and also depends on architectural quantities including layer count, key-value-head count, and head dimension; grouped- and multi-query attention reduce this cost relative to full multi-head attention (Table 1).

Table 1. The four local models on which residency was measured, plus the API executors. This is a selection, not a catalogue: fourteen local models appear in this study, and the full list, with digests, declared context lengths, and per-model default decoding parameters, is Supplementary Table S8. Local models were served by Ollama at 4-bit quantisation, which is not uniform: `gpt-oss:20b` ships MXFP4, while every other local model is `q4_K_M`. We give no residency column because our own residency figures for one of these models do not reconcile; every residency measurement we made, with the discrepancy stated inline, is Supplementary Table S4. We measured residency for four of the fourteen models and not for the other ten: filling that column costs roughly six minutes of cold load per model, and we neither did so nor estimated any figure. Declared parameter counts span 4.0 B (`nemotron-3-nano:4b`) to 36.0 B (`qwen3.6:35b-a3b`).

Executor	Architecture	Total params	Active params/token	Quantisation	Role
<code>qwen3.6:27b</code>	dense	27 B	27 B	<code>q4_K_M</code>	benchmark 1;
<code>qwen3.8:27b</code>	dense	27 B	27 B	<code>q4_K_M</code>	error matrix
<code>qwen3.6:35b-a3b</code>	sparse MoE	35 B	3 B	<code>q4_K_M</code>	out-of-sample
<code>gemma4:12b</code>	dense	12 B	12 B	<code>q4_K_M</code>	addition;
<code>claude-opus-5</code> , <code>claude-sonnet-5</code> , <code>claude-haiku-4-5</code>	commercial API	—	—	—	benchmark 2
<code>claude-opus-4-7</code> , <code>claude-sonnet-4-6</code> , <code>claude-haiku-4-5</code>	commercial API	—	—	—	dense arm
					benchmark 1;
					benchmark 2
					MoE arm
					throughput
					measurement
					only
					frontier reference
					gradient
					error matrix

We ran two workflow classes: alignment and small-variant calling, at 7 measured plan steps, and RNA-seq quantification, at 30 measured workflow steps. Four further classes cover most of the remaining routine per-sample work—single-cell RNA-seq, de novo assembly, metagenomic profiling, and ChIP-seq or ATAC-seq peak calling—and we ran none of the four. Which two classes we ran is the single most important limitation in this paper. The two classes run here are the two in which the plan can state the answer; the four not run are ones in which it cannot. In each of the four, at least one parameter is a scientific judgement made

against the data: filtering thresholds, clustering resolution, assembler parameterisation, database version, peak-calling thresholds. There is thus no canonical value for a plan to carry and no invocation for an executor to transcribe. The plan-sufficiency result is therefore conditional on tasks with no data-dependent parameter choices, and it may be in part a restatement of the task-selection criterion (Supplementary Table S19).

Two limits apply to all results. First, the series of plans at increasing levels of detail (the plan gradient) is exploratory: we wrote the most detailed plan after we had observed failures with the shorter plans, not before. Second, benchmark 2 is a case study, and we do not combine results from the two benchmarks. The detailed plan also creates a design confound. The meta-prompt that produced it requests every flag with its exact value and states that the executor will copy the commands into a shell script, so success with this plan can include direct transcription. We measured transcription in two ways. A token-recall index gives an indirect measure (Results, *How much of the plan the models copy*), and the moved-path test gives a direct measure (Results, *Moving one path separates transcription from binding*). Results below present the plan series for benchmark 1, then the copying and repair tests, then the Galaxy case study.

Methods

Extended methods are in the Supplement.

Study design

We ran the two benchmarks on different harnesses, models, scoring methods and infrastructure, but held one division of labour constant: a commercial API model authored the plan before execution, and a local 4-bit open-weight model executed it (Figure 1). Benchmark 1 gave the executor one prompt and collected one shell script, so the model could not observe execution and could not retry. Benchmark 2 gave the executor multiple tool calls; it read state from a public server and kept state between sessions.

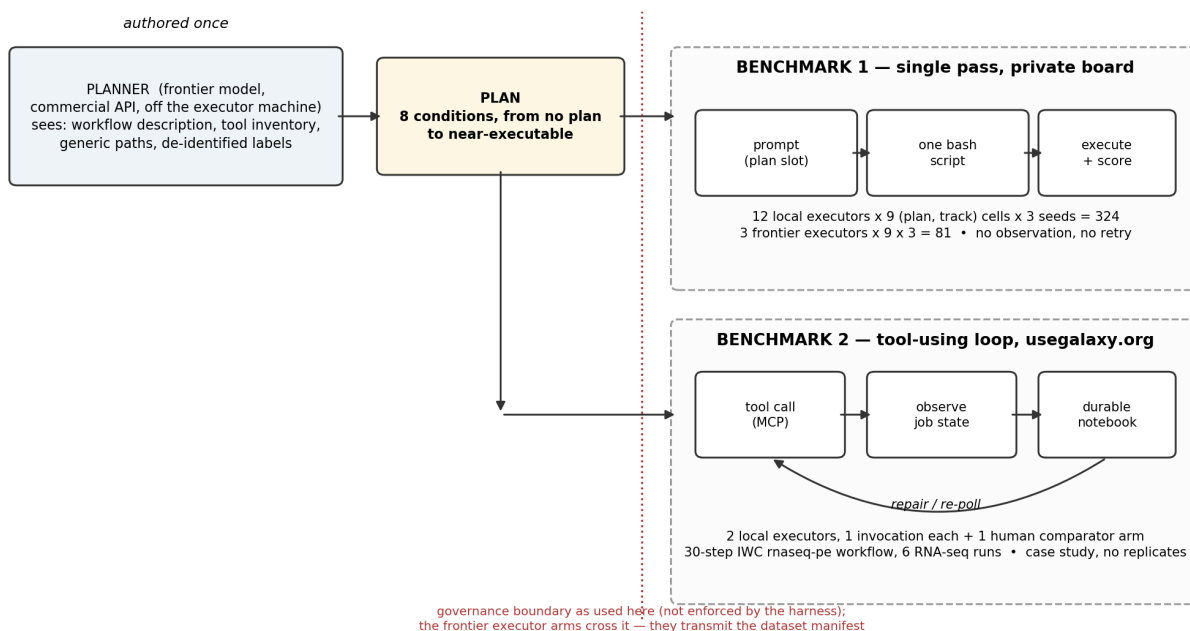


Figure 1: Study design

Figure 1. Study design. A plan is authored once, off the executor machine, by a frontier model in the planner role. Benchmark 1 executes that plan in a single pass: one prompt in, one bash script out, executed and scored with no observation step. Benchmark 2 executes it in a tool-using loop against the public usegalaxy.org server, with observation, durable state and repair. The dotted line marks the data-governance boundary as it

was used here. The harness does not enforce that boundary, and the frontier executor arms of benchmark 1 cross it because their prompts carry the dataset manifest.

The initial benchmark-1 design comprised 324 local generations and 81 API generations: 12 local executors and 3 API executors, crossed with 9 plan-file and track combinations, at 3 seeds each. The nine combinations represent seven plan conditions, because Track B occurs three times. Ten released logs contain 848 cells — a cell is one executor, plan condition and seed — and 75.27 h of generation. API calls account for 81 cells and 1.55 h (Supplementary Tables S1–S2). Three later passes, which sit outside the 848-cell ledger, add 351 local cells. The first is a 273-cell extension that raised selected conditions from three seeds to ten. The second is a 39-cell pass in which the detailed plan was left verbatim while the reference genome was moved (described below). Under that move, a model that copies the plan’s paths fails, while a model that reads the prompt’s paths succeeds. The third is a 39-cell pass that re-ran that moved-path condition allowing up to three attempts, with the failing script and its error returned to the model after each nonzero exit. The repair log has one duplicate skipped row, which we did not count as a cell. We discarded 400 earlier runs because they used a `q8_0` key-value cache. Supplementary Table S10 gives generation outcomes for each block.

Benchmark 1: task, plan conditions, and execution

The task was per-sample mitochondrial variant calling on four deeply downsized paired-end Illumina amplicon samples, drawn from blood and cheek tissue of a mother-child pair, with nine truth variants per run. The reference workflow indexes the 16,569-bp chrM reference with `bwa index` and `samtools faidx`, aligns with `bwa mem` piped to `samtools sort`, and indexes the resulting BAM. It then calls variants with `lofreq call-parallel`, compresses and indexes the VCF with `bgzip` and `tabix`, and finishes with a `bcftools query` plus `awk` fan-in that collapses all four samples into one `collapsed.tsv`. Only this fan-in couples samples to one another (Supplementary Figure S1).

We ran executors under eight conditions, spanning no plan at all to a near-executable recipe (Table 2). Six conditions are plan files; Track B and v0.5 are prompt-template variants that substitute no plan file. The plan files were produced by three planner meta-prompts — the prompts we gave the planner model to make it write each plan. The meta-prompts are reproduced verbatim in the Supplement, together with the executor prompt template. Two pieces of content that the results depend on originated in the human meta-prompt, not with the planner model. The first is the literal read-group string, which `PLANNER_PROMPT.md` requires reproduced “EXACT[ly], character-for-character”; it is therefore present even in the lean v1 condition. The second is the naming of `lofreq call-parallel`, `bgzip` plus `tabix` and `bcftools query`. The gradient is therefore in part a gradient in how much literal syntax was demanded of the planner, not purely in model-authored detail. The exception to model authorship is v1g: its `lofreq` block was extracted mechanically from the Galaxy IUC tool registry (tools-iuc commit 39e7456), and we included it as a check on sensitivity to plan authorship, not as an arm of a planner comparison.

Table 2. Plan conditions. Sizes are bytes of the substituted plan text, or of the prompt-template variant for the two no-plan conditions.

Condition	Size	Summary	Hypothesis tested
Track B	1.2 KB	Problem statement and tool inventory only	How much does a model do unaided?
v0.5	1.4 KB	Track B plus one line giving the tool order	Does sequencing alone help?
v1 (lean)	3.1 KB	Numbered bullets naming tools and key flags; no code	Reference lean plan
v1.25	3.1 KB	v1 plus one literal <code>lofreq call-parallel</code> command line	Does one full command bridge v1 to v2?
v1.5	1.3 KB	v2 with all prose stripped; code fences only	Does the prose change outcomes, or is it decoration?

Condition	Size	Summary	Hypothesis tested
v1g	4.2 KB	v1 plus a <code>lofreq</code> invocation extracted mechanically from the Galaxy IUC tool registry	Can a tool registry substitute for a plan author?
v2 (detailed)	4.6 KB	Exact command line and a Gotchas block per step	Reference detailed plan
v2_defensive	6.5 KB	v2 plus <code>try()</code> helper, per-step validation, retry-once, failure log	Does error-handling prose change runtime behaviour?

Every Track-B run is a no-plan run, regardless of which plan file was nominally passed on the command line. Pooling by plan file without honouring this yields the convincing but false conclusion that the most detailed plan performs worst; `revision/scripts/gradient.py` implements the correct mapping and asserts it.

All local generations ran on one NVIDIA Jetson AGX Orin developer kit: 64 GB of unified LPDDR5 at 204.8 GB/s, released 2022. This memory class is now ordinary desktop hardware: the Mac mini announced in August 2026 reaches the same 64 GB of unified memory, at 307 GB/s, in its M5 Pro configuration (Apple 2026). The comparison is of memory capacity, bandwidth and price class only; no Apple machine was run in this study. All local generations were served by Ollama 0.32.5 at `temperature = 0.2` with `num_ctx = num_predict = 16384`. Neither the local seed nor the API run identifier gives bitwise determinism, so all claims of reproducible sampling have been removed. One serving decision is a confound, not merely a disclosure: the harness overrode only `temperature`, leaving `top_k` and `top_p` at each model’s own defaults. Those defaults are not uniform across families, and are absent altogether for `qwen3.8:27b` (Supplementary Table S8). Every cross-model comparison here therefore compares models decoded from different distributions. The gradient itself is a within-model comparison and is unaffected.

Generated scripts were executed on the model host, in a per-cell working directory, under a pinned conda environment, and killed at 600 s. Execution had no human review, and no container, VM, seccomp profile, namespace, cgroup quota or egress restriction: only the wall-clock kill, the PATH-restricted inventory and the working directory are enforced. The constraints on absolute paths, package managers and network fetches are prompt instructions a model is free to violate, so we measured the violation rate in the Results rather than asserting compliance. The local arm does not meet the sandboxing bar a production deployment needs and is not a deployment template. Attacker- or accident-controlled text reached the executor through at least five unsanitised channels. No indirect-prompt-injection test was run.

Scoring, error injection, and interval estimates

We assessed success on a three-rung ladder whose names match the released scorer’s field names. The first rung, M1 (`m1_executes`), asks whether `bash run.sh` returned 0 within 600 s; the second, M2 (`m2_schema`), whether every expected output exists and `bcftools` accepts every VCF; the third, M3 (`m3_jaccard`), whether the per-sample variant output matches the answer key. Two properties of M3 govern how the Results read. First, M3 depends on exit status: `score_run.py` sets it to 0.0 whenever the run exited non-zero, regardless of what is on disk. Second, M3 is the tolerant Jaccard overlap rather than a per-sample success fraction: it is the per-sample Jaccard of (chrom, pos, ref, alt) against the answer key, with allele frequencies matched within ± 0.02 , averaged over four samples. On the 324 local reasoning-off cells it takes only 0.0 and 1.0; on the frontier arm it does not. Conventional variant-calling metrics were computed alongside, not instead (Supplementary Table S12), and the departures from the nearest published conventions (Krusche et al. 2019) are enumerated in Supplementary Table S5.

We injected tool failures with PATH shims applied only to the targeted tool, so the generated script saw nothing but ordinary exit codes, stderr and output files. Each executor–plan combination comprises 39 cells: 8 pattern–target combinations injecting a true error $\times$ 3 seeds, 4 injecting no error $\times$ 3 seeds, and 3 uninjected controls. Two of the seven patterns do not inject an error at all — `slow_tool` sleeps 30 s and then succeeds, and `stderr_warning_storm` emits 200 warnings and then succeeds — so a third of the injected matrix tests latency and log-noise tolerance, not recovery. We therefore report results by class and never pool them (Supplementary Table S6).

The full gradient was run at $n = 3$ seeds per cell. The headline condition (plan v2, Track A — the arm whose prompt carries the plan file, where Track B carries none — with reasoning off) was raised to $n = 10$ by adding seven seeds and pooling. After the second review cycle we likewise raised the three discriminating conditions — v1g, v1.25 and v1.5, Track A, reasoning off — to ten seeds per model across thirteen local executors. The thirteen are the twelve gradient models plus **qwen3.8:27b**; that model’s original three seeds sit in a separate log and are not pooled, so it contributes seven. One attempted v1g cell has no score record in the released log and could not be reconstructed, so we treated it as missing rather than as a failed generation. The denominators are 126 for v1g and 127 for v1.25 and v1.5 (Supplementary Table S16); treating the missing cell as a failure gives 31/127 for v1g and does not change the conclusions. This is repeated sampling from a single configuration, not replication: ten seeds on one machine, one engine build, one quantisation, one plan and one scorer vary the sampler and nothing else. Generation attempts ending in truncation, refusal or provider error are scored 0 and retained; no completed generation was dropped for producing a bad answer. All success proportions carry Clopper–Pearson exact 95% confidence intervals, and the intervals are wide here: 3/3 gives [0.29, 1.00], 10/10 gives [0.69, 1.00].

The perturbed-plan condition

We added one condition after the second review cycle to separate transcription from binding directly — copying the plan’s command lines as written, against binding them to the inputs stated in the prompt. The plan is v2 verbatim, not one character changed. The sandbox holds the same four read pairs, but the reference genome sits at `data/ref/GRCh38_chrM/rCRS.fa`, and the plan’s literal path, `data/ref/chrM.fa`, does not exist. The prompt’s DATASET section states the real path, so every fact needed to succeed is in the executor’s context. A script that copies the plan’s command lines fails at the first step; a script that binds them to the stated inputs succeeds. We validated the sandbox before any model ran, with a hand-written script of each kind: the copying script fails at `bwa index`, and the binding script produces the correct call set. Thirteen local executors — the twelve gradient models plus **qwen3.8:27b** — ran at 3 seeds each, reasoning off, Track A, giving 39 cells (`matrix_jetson_perturbed.jsonl`; per-cell artifacts under `revision/runs_perturbed/`). The control is the unperturbed published v2 condition, 34 of 36 perfect across the twelve gradient models. No frontier executor was run on the perturbed plan, because no API spend was authorised for this pass.

Benchmark 2 and the fabrication study

The second benchmark re-quantified *Candidozyma auris* RNA-seq from Santana et al. (2023), BioProject PRJNA904261, six paired-end runs SRR22376027–SRR22376032. We chose this target because it is a published benchmark with known expected values (~5,594 genes, ~92% assigned). Relative to benchmark 1 it varies four things at once: a second workflow class, a shared production server, a 30-step workflow, and an external cost comparator (Nekrutenko 2026). The executor-selection rule is invoked repeatedly in what follows, so we state it verbatim:

take every local model that scored 3/3 at plan v2 Track A with reasoning off in the benchmark-1 gradient, and from that set take the largest dense model and the largest mixture-of-experts model by total parameter count.

The rule returns **qwen3.6:27b** and **qwen3.6:35b-a3b**. The dense arm actually run was **qwen3.8:27b**, substituted at matched parameter count after the rule was fixed, and that arm is therefore exploratory rather than confirmatory. Plan sufficiency was never confirmatory on either benchmark, because the Galaxy plan was corrected repeatedly while benchmark 2 ran.

Both executors ran on one Jetson AGX Orin 64 GB, with reasoning suppressed, under `galaxyproject/loom`, with Galaxy reached over MCP and `notebook.md` serving as durable inter-session state. The plan was a router — a 50-line index plus seven step files — so the executor loads only the step it is on. Its seven steps ran from importing the workflow by TRS identifier to reporting the per-sample counts. No number reported for this benchmark comes from an agent’s own report: we read every state, parameter and count back from the Galaxy API or from dataset contents, because one executor produced a fully formed failure report for an event that never occurred. Scoring had four components: 1) invocation correctness, field by field; 2) biological outcome against the published benchmark, read from dataset contents; 3) end-to-end reporting, meaning

whether the executor itself delivered the per-sample table with a dataset identifier beside every number; and 4) a defect ledger attributing every human intervention to the planner or the executor side.

One change belongs in Methods because it changed the design. The original step-7 verification file stated its own expected values, which let an executor that never opens a dataset verify by recitation. We rewrote the file to state no expected values and to sanction **UNAVAILABLE**. The general rule follows: any agent evaluation that places the expected answer in the task description cannot distinguish execution from recitation.

The fabrication incident was a single observation, so we ran a controlled follow-up to bound the rate. The design was 3 conditions $\times$ 2 models $\times$ 5 seeds, one fresh workspace per cell, a fixed prompt and known ground truth. The task was to fetch six featureCounts summary datasets from a real Galaxy history and report each **Assigned** value with the dataset identifier it was read from. The conditions were blind (nothing said about expected values), leaky (the expected range stated) and blocked (two identifiers replaced with same-shaped ones that do not exist, so **UNAVAILABLE** is the only correct answer). Answers were scored **CORRECT**, **WRONG** — the fabrication category — **ABSTAINED** or **ABSENT**. The design is 30 launches, of which 24 produced a scorable answer file, giving 144 sample-level judgements; six, 20%, produced no answer to score. Three limitations govern the rate. First, judgements within a session are not independent, so the session is the unit of analysis. Second, unscorable launches are excluded, so the rate is fabrication conditional on an answer having been produced, and the excluded class is structurally the one the original incident belongs to. Third, every cell is single-shot with a working tool path, whereas the incident arose in a resumed session with no reachable path.

Cost model, data governance, and availability

Two of the cost model’s eight inputs are measured. Frontier-API cost was \$0.0662 per run over 81 API cells, and Jetson generation time, verified from the released log, is 80.3 s per run (26,010 s over 324 cells). The cost model uses 86.7 s per run — 7.8 h recorded for the block at run time, including model loading, execution and scoring — but that block figure was not archived and is carried as a stated input, not a verifiable measurement. Substituting the verified 80.3 s changes the electricity term by 8% and no conclusion. The other six inputs are assumptions, exposed as parameters in `revision/scripts/cost_model.py` (Supplementary Table S9). The model reports a like-for-like annualised comparison, with an API-side labour term exposed rather than set to zero. Supervision labour is not priced at all, and benchmark 2 shows it is not zero; the cost model is therefore the zero-touch case, which benchmark 2 did not achieve.

The plan-then-execute split also defines a data-governance boundary, and it was respected by the planner role only. The frontier executor arms — 315 API cells in all — transmitted the executor prompt, which carries the dataset manifest with this study’s own subject labels. `revision/scripts/payload_audit.py` reconstructs every outbound payload and scans it against a deny-list of local-identity tokens (Supplementary Table S3). Benchmark 2’s MCP traffic is neither archived nor reconstructable, and it is outside the audit.

The repository at <https://github.com/nekirut/LLM-eval-paper> is released under an MIT license. It contains the plan files and prompt templates, the planner meta-prompts, the harness and sweep drivers, the scoring code, every archived `run.sh` and per-cell artifact, the error-matrix logs, and the revision analysis scripts. Four items are not released: the Galaxy plan files, the fabrication study’s code and per-cell outcomes, the eleven-item defect ledger with its diffs, and the `loom` session logs. Benchmark 2’s per-sample counts and gene count are re-readable from public Galaxy histories, but its defect count and attribution, its tool-call and wall-clock figures, and the fabrication study’s cell counts rest on `revision/galaxy_demo/FACTS.md`. That file is a hand-typed contemporaneous record of live API read-back — the same evidentiary class as an assertion in this manuscript — and those figures are marked as such wherever used.

Results

The two executor classes diverge most where the plan is thinnest

We began by measuring how much written guidance each executor class needs. We ran every executor against the plan gradient, from no plan at all to a plan spelling out every command (Figure 2, Table 3). With no plan, 6 of 108 local runs produced the correct call set (exact 95% CI [0.02, 0.12]), while the frontier executors produced it in 13 of 27 runs ([0.29, 0.68]). Plan v1, the lean plan, contains no command lines, yet the frontier

executors were correct in all 9 v1 runs and in all 36 runs with the four more detailed plans. Thus more plan detail did not increase their observed success proportion, although the interval on 9/9 is wide (exact 95% interval [0.66, 1.00]). The local executors reached 34 of 36 correct runs only with v2, the most detailed plan.

The tested executor classes therefore differed in how much specification they needed, with the largest contrast where the plan was thinnest — a contrast our design cannot attribute to any single variable. The local arm is 4-bit quantised, at most 35 B, open-weight and served by Ollama on our own hardware. The frontier arm is unquantised, of undisclosed size, from a single vendor, served over an API and decoded under different sampling parameters. These variables move together, and this design separates none of them. Quantisation in particular is a live alternative explanation for a model that cannot emit a correct `lofreq call-parallel` flagset. We did not run the decisive control: `qwen3.6:27b` at `q8_0` or `fp16` against its own `q4_K_M`. If precision closes a meaningful part of the gap, the finding is about quantisation rather than open weights. Three models from one vendor is also not a sample of frontier models.

Figure 2. Plan gradient with reasoning disabled, Jetson AGX Orin, $n = 3$ seeds per cell. Rows are the twelve benchmark-1 local executors plus the out-of-sample addition `qwen3.8:27b`, thirteen rows in all. The twelve-model gradient totals given in the text exclude `qwen3.8:27b`, which was run separately and is shown here for comparison. Columns run left to right from no plan (Track B) to the most detailed plan (v2). Cell values are the mean tolerant Jaccard score (M3), which on this arm takes only the values 0 and 1 per run.

Table 3. Plan gradient, reasoning off, twelve local executors, three seeds per cell. Track B pools three (plan file, track) cells, hence $n = 108$. Intervals are Clopper–Pearson exact 95% intervals. The local arm is reported as a count of runs producing the correct call set, not as a mean score: on all 324 local reasoning-off cells, M3 takes only the values 0.0 and 1.0, so a local “mean M3” column would be the perfect-run proportion written twice and would invite the reader to believe partial biological credit was measured, which on this arm it was not. The tolerant Jaccard is retained only for the frontier arm, where it is non-degenerate: its cells there take the values 0.0, 0.833, 0.938 and 1.0, so a Track-B mean of 0.825 coexists with only 13 of 27 runs perfect. The frontier column pools the three API executors at $n = 9$ per column, 27 at Track B. Three local conditions — v1g, v1.25 and v1.5 — were later raised to ten seeds per model over thirteen executors (31/126, 63/127 and 109/127; Results, *Runnable command syntax, not explanatory prose, carries the plan’s effect*; Supplementary Table S16). The $n = 3$ rows are retained here so that the survey is one design.

Condition	Local runs with the correct call set	95% CI	Frontier runs with the correct call set	95% CI	Frontier mean M3 (Jaccard)
Track B (no plan)	6/108	[0.02, 0.12]	13/27	[0.29, 0.68]	0.825
v0.5 (tool order only)	1/36	[0.00, 0.15]	3/9	[0.08, 0.70]	0.801
v1 (lean)	9/36	[0.12, 0.42]	9/9	[0.66, 1.00]	1.000
v1g (registry- authored)	9/36	[0.12, 0.42]	9/9	[0.66, 1.00]	1.000
v1.25 (v1 + one command line)	15/36	[0.26, 0.59]	9/9	[0.66, 1.00]	1.000
v1.5 (v2 with prose stripped)	30/36	[0.67, 0.94]	9/9	[0.66, 1.00]	1.000
v2 (detailed)	34/36	[0.81, 0.99]	9/9	[0.66, 1.00]	1.000

Runnable command syntax, not explanatory prose, carries the plan’s effect

The gradient bundles prose and command syntax together, so we next pulled the two apart. Plan v1.5 is v2 with all prose stripped, leaving only code fences. It is the shortest plan in the set, at 159 words against v2’s 660, and it is the second best.

The two halves of the prose-versus-syntax contrast are not equally well resolved. Removing the literal command lines, which is what plan v1 is, reduced perfect runs from 34/36 to 9/36. Stripping every word of explanation from v2 reduced them from 34/36 to 30/36, a difference not resolved at these sample sizes. Paired by model, 3 improved, 1 worsened and 8 were unchanged (two-sided sign-test $p = 0.63$). Because three

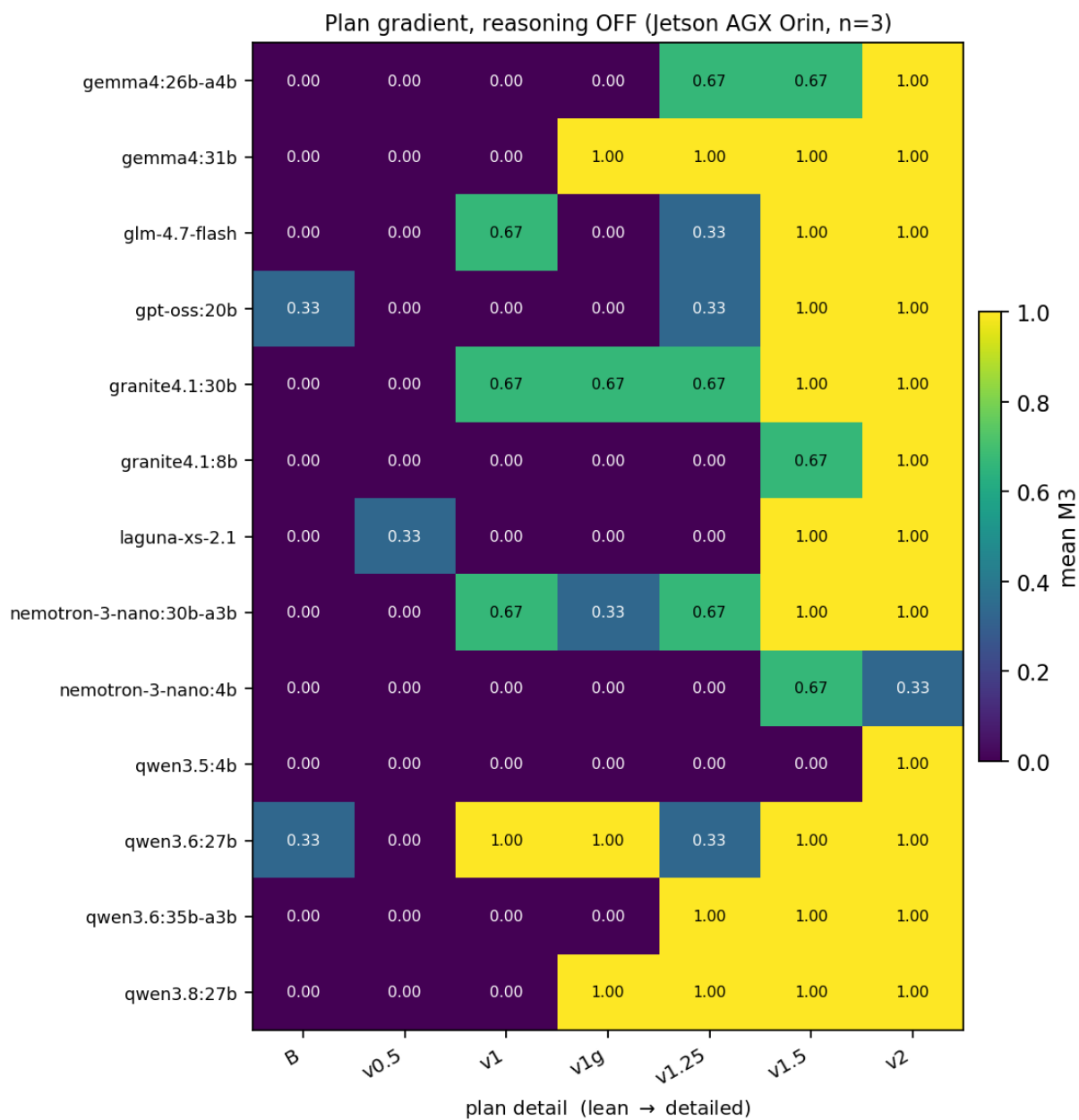


Figure 2: Plan gradient with reasoning disabled

seeds per cell leave that contrast underpowered, we later re-ran three conditions — v1g, v1.25 and v1.5 — at ten seeds per model (the “seed extension” below). The pooled descriptive proportions were 31/126 for v1g (0.246, exact 95% CI [0.17, 0.33]), 63/127 for v1.25 (0.496, [0.41, 0.59]) and 109/127 for v1.5 (0.858, [0.79, 0.91]). Because repeated seeds are clustered within models, we treat the 13 model-level success proportions as paired observations for the primary contrasts. The mean within-model difference was 0.253 for v1.25 minus v1g (exact sign-flip $p = 0.027$) and 0.354 for v1.5 minus v1.25 ($p = 0.0039$). We report the corresponding run-level paired counts in Supplementary Table S16 as a sensitivity analysis. The defensible claim has three parts: runnable command syntax improved outcomes across the tested models, removing prose produced no detectable change, and the size of any prose effect remains uncertain.

The mechanism is more specific than a count of command lines. Plans v1.25 and v1g each add one literal `lofreq` invocation to v1 but present it differently. Plan v1.25 supplies one runnable line: `lofreq call-parallel --pp-threads 4 -f data/ref/chrM.fa -o results/{sample}.vcf results/{sample}.bam`. Plan v1g supplies a six-line block in the registry’s long-form style, with two bare, valueless flags (`--sig`, `--bonf`) that the plan itself says may be omitted. Their pooled success proportions were 0.496 and 0.246, respectively. The v1g plan also states that `results/{sample}.bam` is a positional argument at the end, so the contrast is not explained by omission of that information. Across the 13 paired model-level proportions, v1.25 exceeded v1g by a mean of 0.253 (exact sign-flip $p = 0.027$). This result is consistent with copy-pasteability — whether a command can be pasted and run without modification — rather than information content or verbosity alone, as the relevant plan property. Plan v1.5 is the shortest and the second best, v2 is the longest and the best, and v1g is the second longest but the weakest of the plans containing command syntax (Table 4). We quote no rank correlation over five points with ties.

Table 4. Syntax density against outcome, Track A, reasoning off. “Literal command lines” counts non-blank lines inside fenced blocks that invoke a workflow tool or shell construct. That is an operational definition, and the count is sensitive to it. v1g’s single `lofreq` invocation is written across six physical lines in the registry’s long-form style; it is counted as one, because it is one invocation. v1g’s author is the Galaxy IUC registry, extracted mechanically; every other plan was authored by a frontier model. Jetson values are recomputed under this manuscript’s retention rule, which scores the one truncated v1g generation 0 and keeps it. They therefore differ in the third decimal from `revision/galaxy_demo/SYNTAX_DENSITY.md`, which drops it. Every frontier cell here is 9/9. The ten-seed column pools the seed extension over thirteen local executors (Methods; Supplementary Table S16). v1 and v2 were not extended; v2’s headline pooling to $n = 10$ is Table S11.

Plan	Author	Words	Literal command lines	Local mean M3, Jetson (n = 3/model)	Local mean M3, Jetson (n = 10/model)	Frontier mean M3, API (n = 9)
v1	frontier planner	422	0	0.250 (n = 36)	—	1.000 (n = 9)
v1g	Galaxy IUC registry (mechanical)	558	1	0.250 (n = 36)	0.246 (n = 126)	1.000 (n = 9)
v1.25	frontier planner	413	1	0.417 (n = 36)	0.496 (n = 127)	1.000 (n = 9)
v1.5	frontier planner	159	10	0.833 (n = 36)	0.858 (n = 127)	1.000 (n = 9)
v2	frontier planner	660	8	0.944 (n = 36)	—	1.000 (n = 9)

Across the same five plans, the API executors were correct in 45 of 45 runs while the local success proportion increased from 0.250 to 0.944. Thus, greater plan detail reduced the observed difference between the two executor groups, although the API result is a point estimate from nine runs per plan. The v1g plan is not a planner-comparison arm, because syntactic density was not matched; it is a mechanically extracted plan with a non-model author. Its result shows that authorship and plan length do not explain the observed pattern. The data support a narrower conclusion: local models produced working pipelines more often when the plan contained commands that could be copied without modification. Explanatory prose produced no detectable

effect at the tested sample size. How much of a plan the models copy outright is what we measured next.

How much of the plan the models copy

To measure copying directly, we defined a transcription index: the token-level recall, in an emitted script, of the 90 literal command tokens of plan v2, after normalising the sample placeholder and the thread count. We computed the index over the 787 reasoning-off scripts (Table 5).

Table 5. Transcription index by plan condition, reasoning off, over the 787 archived reasoning-off scripts (the original corpus plus the seed-extension scripts at v1g, v1.25 and v1.5). Recall is of plan v2’s 90 literal command tokens, so all conditions are measured against one fixed reference. Brackets are the interquartile range across scripts. The corresponding bootstrap 95% intervals on the mean have a half-width of at most 0.03 in every cell with $n > 9$, and are omitted for width. The corpus includes the 48 `qwen3.8:27b` scripts that Table 3 excludes; they are 6% of it.

Condition	Local, all scripts	n	Frontier, all scripts	n	Local, correct runs only	n	Frontier, correct runs only	n
Track B	0.446	117	0.520	27	0.428	6	0.505	13
(no plan)	[0.42–0.49]		[0.47–0.56]				[0.46–0.54]	
v0.5	0.454	39	0.501	9	0.467	1	0.444	3
	[0.42–0.49]							
v1	0.515	39	0.522	9	0.509	9	0.522	9
	[0.48–0.54]							
v1g	0.520	129	0.517	9	0.505	34	0.517	9
	[0.48–0.56]							
v1.25	0.516	130	0.533	9	0.513	66	0.533	9
	[0.49–0.54]							
v1.5	0.702	130	0.658	9	0.705	112	0.658	9
	[0.68–0.74]							
v2	0.746	122	0.695	9	0.749	110	0.695	9
	[0.74–0.76]				[0.74–0.76]			

The 13 correct API runs with no plan had a mean transcription index of 0.505, while correct local runs with v2 had a mean index of 0.749. Thus approximately half of the v2 command tokens occur in a correct pipeline without plan conditioning, and plan conditioning added approximately one quarter of the tokens. The index did not predict success within a plan condition. At v2 the means were 0.749 for successful scripts and 0.722 for failed scripts, and at v1 they were 0.509 and 0.517. Local scripts recovered 0.942 of the tokens in the code-only v1.5 plan ($n = 130$).

What the index cannot settle is what the remaining quarter is. A model that merely reflows the supplied tokens — call it a transcriber — and a model that reconciles those tokens against the actual inputs — a binder — both score near 0.75. No token count forces them apart. The condition that does force them apart is a perturbed plan, v2 verbatim with the reference moved, because a transcriber reproduces the defect and a binder repairs it. We ran that condition after the second review cycle; it is the next section.

Moving one path separates transcription from binding

We detail the design in Methods. The plan is v2 verbatim, but the reference genome sits at `data/ref/GRCh38_chrM/rCRS.fa` instead of the plan’s literal `data/ref/chrM.fa`, and the real path is stated in the prompt’s DATASET section. Before any model ran, we validated the sandbox with two hand-written scripts: one copies the plan’s path verbatim and must fail, the other binds to the stated path and must succeed. We then ran thirteen local executors at three seeds each, against a control of 34/36 perfect on the unperturbed v2.

The result is 6 of 39 perfect, and the outcome is bimodal by model: every model is 3/3 or 0/3 (Table 6). We verified this from the emitted scripts rather than inferring it from the scores. Both surviving models wrote `data/ref/GRCh38_chrM/rCRS.fa` throughout. All eleven failing models wrote the plan’s `data/ref/chrM.fa`

verbatim — several derived index names such as `chrM.fa` from it — and every failing run died at the first step, with `bwa index` unable to open the file.

Table 6. Plan v2 verbatim with the reference moved to a path stated in the prompt, thirteen local executors $\times$ 3 seeds, reasoning off. Outcomes read from the emitted scripts and per-cell logs (`matrix_jetson_perturbed.jsonl`; artifacts under `revision/runs_perturbed/`). Control: unperturbed v2, 34/36 perfect.

Outcome	Models	What every emitted script wrote
Bound (3/3 perfect)	<code>gemma4:31b</code> , <code>gpt-oss:20b</code>	the true path <code>data/ref/GRCh38_chrM/rCRS.fa</code> , carried into every derived path
Copied (0/3 perfect)	<code>qwen3.6:27b</code> , <code>qwen3.8:27b</code> , <code>qwen3.6:35b-a3b</code> , <code>gemma4:26b-a4b</code> , <code>glm-4.7-flash</code> , <code>granite4.1:30b</code> , <code>granite4.1:8b</code> , <code>laguna-xs-2.1</code> , <code>nemotron-3-nano:30b-a3b</code> , <code>nemotron-3-nano:4b</code> , <code>qwen3.5:4b</code>	the plan's stale <code>data/ref/chrM.fa</code> verbatim; every run failed at the first step (<code>bwa index</code> : fail to open file)

Four things follow, and together they change what the gradient means. First, for eleven of thirteen local models, v2 success is transcription: they copy the plan's command lines without reconciling them against the stated inputs. One path mismatch, stated in the prompt they were reading, sent a condition that scores 34/36 to 0. Reviewer 2's objection — that the headline result rewards pasting — is therefore confirmed for most of the model class, as a measurement rather than a concession. Second, binding exists in this model class but is rare. Two of thirteen models adjusted every path and produced the correct call set in all three seeds, so binding is demonstrable rather than hypothetical — a property of individual models, not of the class. Third, rank on the unperturbed gradient does not predict binding. The paper's original headline model, `qwen3.6:27b`, 10/10 at unperturbed v2, copied, while `gpt-oss:20b`, one of only three models below 10/10 at the unperturbed headline condition (9/10), bound. Fourth, the claim that survives is narrower than plan sufficiency: a detailed plan makes most local models work only while its literal paths match the data. Reliable execution under a stale plan requires either a binding-capable model or a plan kept exactly current. Whether frontier executors bind under the same perturbation is unknown, because we ran no frontier arm. And at three seeds per model, a 3/3 is [0.29, 1.00], so the bimodality is sharp but each per-model rate is wide.

Three attempts separate transcribers that repair from transcribers that do not

The perturbation above gave each model one attempt: it wrote a script, the script ran, and that was the end of the trial. We therefore re-ran it with feedback, keeping the perturbed plan unchanged and allowing up to three attempts. After any nonzero exit the model saw its own script, the exit code and the last 40 lines of the execution log, and could submit a fix. The retry signal is the exit code only; the score is computed afterward and never shown. We ran thirteen local executors at three seeds each (`matrix_jetson_repair.jsonl`; per-attempt scripts under `revision/runs_repair/`).

Feedback split the eleven models that had copied into models that repair and models that do not (Table 7; per-seed attempts and scores are Supplementary Table S18). Three of the eleven repaired. Eight did not, although the error named the missing file and the correct path was in their prompt. Repair rescued 8 of the 33 seeds from the copying models, so with repair the class reached 14 of 39 perfect runs, against 6 of 39 one-shot and 34 of 36 unperturbed. The one-attempt constraint therefore accounts for part of the transcription finding, not all of it. The taxonomy is three tiers: bind (2 models), repair (3), neither (8). One retry converted `qwen3.6:27b` and `qwen3.8:27b` from total failure to perfect: for those two models, an execute-and-retry round was worth more than any amount of plan prose.

Table 7. The moved-reference condition re-run with up to three attempts, the error fed back after each nonzero exit. Thirteen local executors $\times$ 3 seeds, reasoning off. Per-model attempts and per-seed scores are Supplementary Table S18.

Tier	Models	Outcome
Bind on attempt 1	<code>gemma4:31b</code> , <code>gpt-oss:20b</code>	one attempt, perfect, all seeds (the controls; unchanged)
Repair on attempt 2	<code>qwen3.6:27b</code> , <code>qwen3.8:27b</code>	perfect on attempt 2 in all three seeds
Partial repair	<code>laguna-xs-2.1</code>	perfect on attempt 2 in two of three seeds
No repair	the remaining eight models	0 of 24 seeds recovered in three attempts

We verified three behaviours from the per-attempt scripts rather than inferring them from the scores. First, the fix by `qwen3.6:27b` is a genuine repair: its second attempt tests for the old path, falls back to the path stated in the prompt, and proceeds. It used the prompt’s information once the error pointed at the file. Second, `qwen3.6:35b-a3b` resubmitted the identical dead path in all three attempts, with the error naming that path in front of it. Third, `granite4.1:30b` and `gemma4:26b-a4b` guessed `data/ref/rCRS.fa` on attempt 3 — the correct filename from the prompt, in the wrong directory — so they used part of the prompt’s information and did not check the rest. Failure to repair is not failure to act: the non-repairing models changed other things — flags, loops, index names — while keeping the dead path. Four caveats bound this arm: three seeds per model, one perturbation type, a retry only on a nonzero exit, and no frontier arm.

The $n = 3$ headline does not survive repeated sampling

Because three seeds per cell leave wide intervals, we added seven seeds to the original three at v2 Track A (Figure 3, Supplementary Table S11). The headline moved. Nine of 12 executors are 10/10 (exact 95% CI [0.69, 1.00]); `gpt-oss:20b` is 9/10; `qwen3.5:4b`, which scored 3/3 at $n = 3$, is 7/10; and `nemotron-3-nano:4b` is 1/10 ([0.00, 0.45]). Every model that moved, moved down — the expected direction rather than symmetric noise — so we report $n = 10$ as the headline and treat every $n = 3$ cell as an interval. The out-of-sample pair `qwen3.8:27b` against `qwen3.6:27b` is underpowered rather than null: 8 of 27 matched cells are discordant, splitting 6 to 2 in favour of the *older* model ($p = 0.29$). We have therefore removed the generational framing from the abstract and the conclusion.

Figure 3. Repeated sampling with plan v2 (plan v2, Track A, reasoning off). Grey points are the $n = 3$ means as originally published; red points are the pooled $n = 10$ means. Every model that moved, moved down; no model rose. This is repeated sampling from one configuration, not independent replication.

The conventional metrics are not degenerate: recall without the exit-status rule sits above run-level success

The conventional variant-calling metrics are worth reporting, but only with their construction stated. Over all 324 reasoning-disabled runs, TP = 1,042, FP = 0 and FN = 1,874; precision is 1.000 by construction, because the truth set is the canonical output of the same deterministic caller. Recall, however, does not equal the run-level success rate in any condition except v2. It is 0.086 against 6/108 at Track B, 0.306 against 9/36 at v1, 0.485 against 15/36 at v1.25, and 0.886 against 30/36 at v1.5 (Supplementary Table S12). Pooled, recall is 0.357 against a success rate of 0.321. We report no pooled F1: it averages over conditions that share no denominator of interest, and it re-expresses a binary.

The recall curve computed without the exit-status rule sits above the run-level success curve at every lean condition, and the excess is real biology: variants correctly called by runs the run-level metric scores 0. Two mechanisms produce it. Of the 324 runs, 26 produced exactly 2 of the 9 truth variants, concentrated at the lean end, so output is near-bimodal but not strictly so. In addition, 8 of the 324 wrote all four VCFs and then exited non-zero, of which 6 had recovered all nine truth variants. That is why 110 runs recovered the complete truth set while only 104 scored a perfect run. Scoring such a run 0 is defensible — a pipeline that exits non-zero has not finished — but it is a choice. That choice, not the biology, is what makes the run-level metric degenerate. So the conventional metrics quantify how often a model gets the biology right and the plumbing wrong: 8% of runs by partial call sets, plus 2.5% by exit status. What they cannot do is separate a good caller from a bad one, because the truth set is the same caller’s own output and we ran no perturbation series that would vary accuracy continuously.

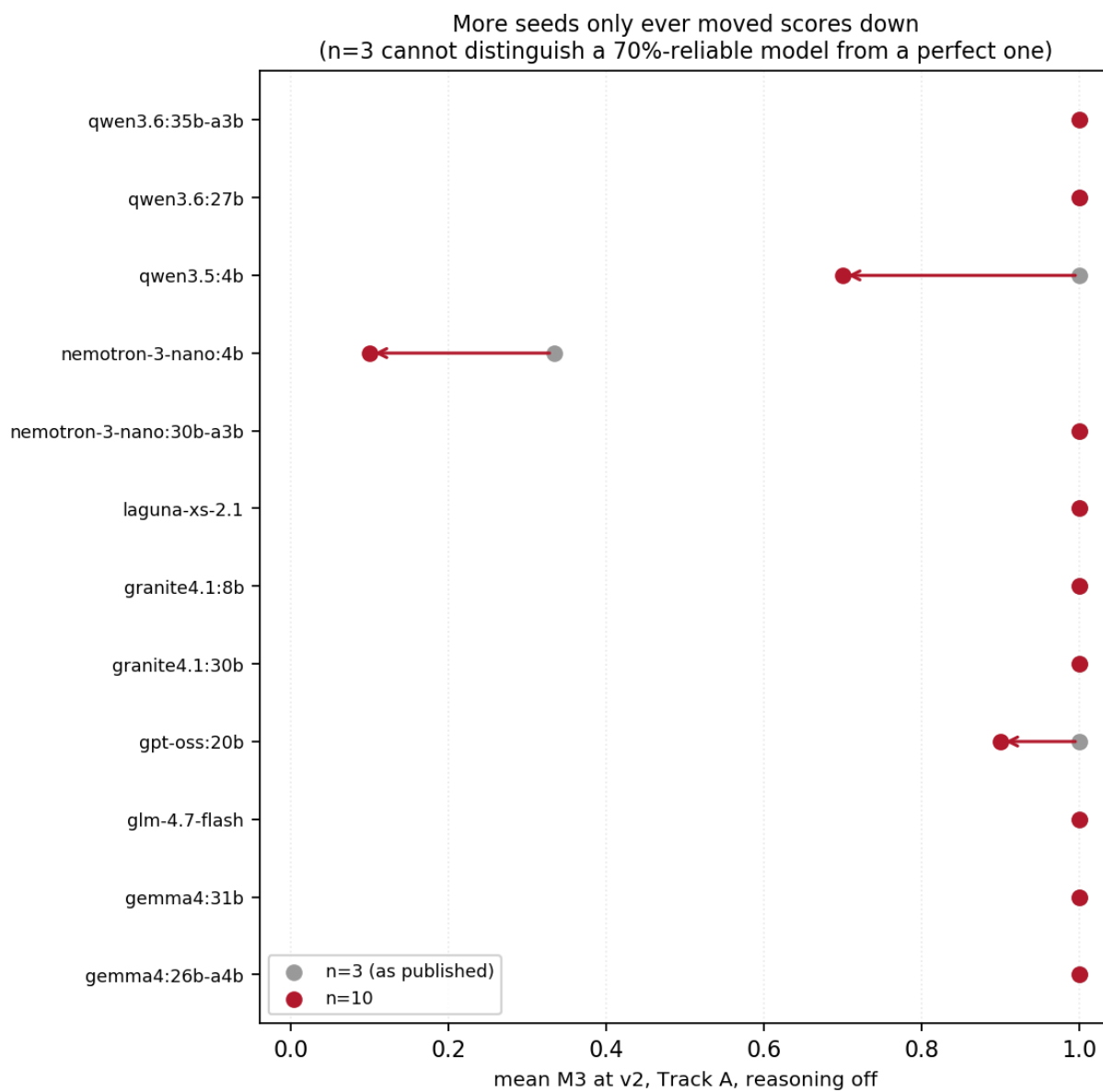


Figure 3: Repeated sampling with plan v2

Enabling reasoning does not substitute for plan detail

We next asked whether enabling reasoning — letting a model emit a chain of thought before its script — substitutes for plan detail. It does not. Over 324 reasoning-disabled and 270 reasoning-enabled cells, restricted to the 10 models runnable in both arms, reasoning helps where the plan is thin and costs accuracy where it is sufficient. The per-plan differences are +0.17 at v1, +0.23 at v1.25 and −0.20 at v2, and at v2 Track A, 4 of 10 models regress while none improve. Enabling reasoning on **qwen3.8:27b** at plan v1g also needs a budget two to three times larger merely to emit a script. Even then the model falls from 3/3 correct in ~130 s to 1/3 correct in 7,395–12,974 s. Thus chain of thought is not a substitute for a sufficient plan, and at the sufficient plan it is a liability. The reasoning-enabled arm also ends at the harness’s own limits in 6.7% of runs against 0.3% disabled, so it measures model-plus-budget rather than model capability (Supplementary Results, Figures S2–S3).

Injected failures: the plan and the injected pattern, not the model, set the outcome

The injected-failure matrix asks what a model does when a tool breaks mid-pipeline. For **qwen3.6:27b** with plan v2, the 24 cells containing a true tool failure produced 15 crashes, 6 propagated errors, and 3 recoveries (Figure 4). The recovery rate was 12.5% (exact 95% CI [0.03, 0.32]), the crash rate 62.5% ([0.41, 0.81]), and the mean M3 0.000. The other 12 cells did not contain a tool failure, so results that pool the two classes are not recovery rates. These runs gave the model one attempt and could not retry; we tested retries separately, under the moved-path condition (*Three attempts separate transcribers that repair from transcribers that do not*).

The identical distribution obtained for the frontier executor is a degeneracy, not a parity finding. Across the 39 v2 cells, **qwen3.6:27b** and **claude-opus-4-7** agree in every cell on both M3 and the handle category — crash, propagated error or recovery. The same holds under v2_defensive, and five of the six executors produce exactly crash 15 / propagate 6 / recover 3 on the 24 true-error v2 cells. Two architecturally unrelated families do not converge cell-for-cell by shared competence, so the handle category at the baseline plan is essentially determined by the injected pattern. Two things bound that reading. The per-cell artifacts for the error matrix were not archived, so the script diff that would confirm it cannot be run. The frontier comparator here is also a different model generation from the arm that produces the paper’s main result. The model term is not zero everywhere. Under v2_defensive, **qwen3.6:27b** and **claude-opus-4-7** resolve the 24 true-error cells to recover 9 and partial 15. **qwen3.6:35b-a3b** resolves them to recover 3 and partial 21 with its mean M3 *falling* to 0.208, and **granite4** produces 24 crashes in 24 cells. So a defensive plan changes error handling, and how much it helps is model-dependent.

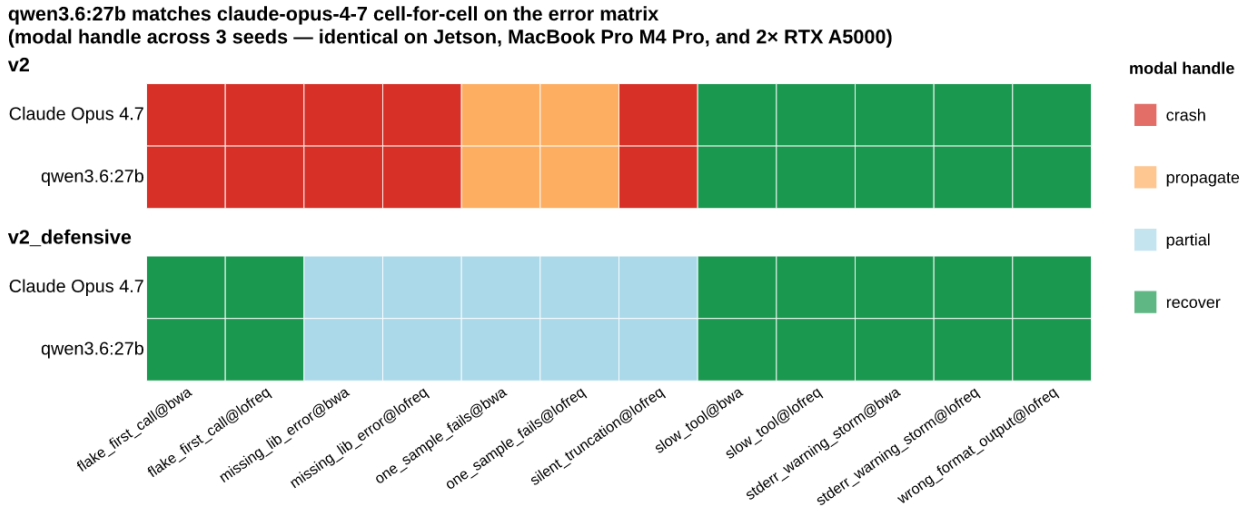


Figure 4: Injected-failure matrix

Figure 4. Injected-failure matrix: modal handle category across three seeds, for the local executor

qwen3.6:27b and the frontier executor claude-opus-4-7. The top block is the baseline plan v2 and the bottom block is plan v2_defensive; the two executors match in all 39 cells of each block. Five of the six executors in the matrix match on the v2 block. That is evidence that the outcome is pattern-determined, not evidence of model parity (see text). Columns 8–11 (slow_tool@bwa, slow_tool@lofreq, stderr_warning_storm@bwa, stderr_warning_storm@lofreq) inject no error; they are latency and log-noise tolerance checks, not recovery tests (Supplementary Table S6). The columns are alphabetical, so the twelfth and rightmost, wrong_format_output@lofreq, is a true-error pattern, and a green cell there is a genuine recovery.

We also audited the 1,039 archived `run.sh` files word by word, replacing the claim that no destructive or escape behaviour was observed with a measurement. None invokes a package manager, network fetch tool or `sudo`, and none directs output to an absolute path outside `/dev/` and `/tmp/`, although 19 (1.8%) write under `/tmp/`. Two scripts perform a genuinely out-of-sandbox action. One, `rm -f /tmp/*.txt /tmp/temp_*.vcf.gz`, came from a no-plan run. The other, `rm -rf /tmp/tmp*`, is an unbounded recursive wildcard delete against a shared directory, and it came from a plan-conditioned run (v1g). The `/tmp/` write rate is higher without a plan — 11 of 303 no-plan scripts against 8 of 736 with a plan — but the strongest single action occurred with one. A plan lowers the rate of this behaviour and does not eliminate the class.

Case study: a tool-using agent loop on a production server

This is a case study, not an experiment: one invocation per arm, no replicates, no frontier control, and a plan repeatedly corrected while it ran, so nothing in it carries a confidence interval. Tool-call and wall-clock figures are transcribed from `FACTS.md`, and the counts are re-readable from the public histories.

Given the plan and a live MCP connection to usegalaxy.org, both local executors imported the 30-step IWC `rnaseq-pe` workflow by TRS identifier, and both verified the six paired-end inputs and the annotation. A human had pre-built both of those after a harness transport fault, so those two plan steps were verification and not construction. Both executors then submitted a parameter-correct invocation: 2 of 2 inputs and 8 of 8 parameters. The dense arm (qwen3.8:27b) did this in 38 tool calls over 44 min, the mixture-of-experts arm (qwen3.6:35b-a3b; the MoE arm below) in 19 over 22 min. The 8/8 figure measures transcription, not derivation, because the invocation step supplied a literal JSON body. The invocations reproduced the published quantification: both arms yielded 5,594 genes, which we verified by downloading and counting the counts table rather than by accepting a reported figure. Assigned fragments were 15.7–25.3 M per sample, at 93.1–93.6% of uniquely mapped fragments, against a published ~92% on that denominator (Supplementary Table S14, Figure 5). The two arms produced byte-identical values from twelve distinct dataset identifiers, which is expected from a deterministic server-side workflow on identical inputs and is not independent replication of model behaviour.

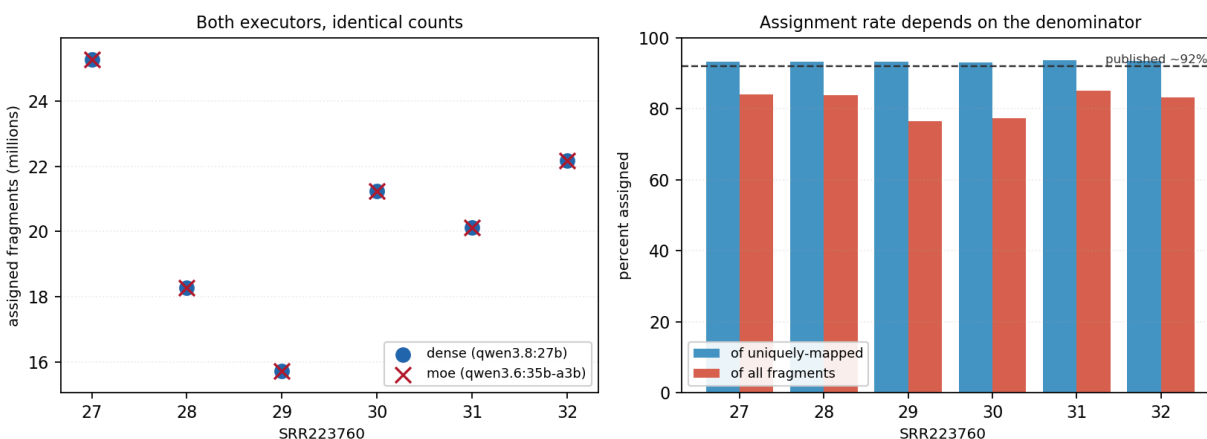


Figure 5: Per-sample featureCounts results

Figure 5. Per-sample featureCounts results for the benchmark-2 case study. Left: assigned fragments per run; the dense (`qwen3.8:27b`) and mixture-of-experts (`qwen3.6:35b-a3b`) arms are exactly superimposed. Right: assignment rate against two denominators — uniquely mapped fragments and all fragments — with the published ~92% benchmark marked. The gap between the two denominators is multi-mapping, a property of the data rather than of the executor.

Three caveats must accompany this result wherever it is claimed. Both executors submitted a parameter-correct invocation, and the workflow produced benchmark-matching counts. But the plan supplied the invocation body literally, a human pre-built two of the seven plan steps’ inputs after a transport fault, and neither executor completed the final reporting step. Neither reported a single per-sample value in the two runs of step 7; a human read the six values out of the server (*Framing, not capability, decides whether the reporting step happens*). “Both executors submitted a correct invocation” is true; “both executors succeeded” is not. Supervised human time was substantially larger than the agent wall time and was not logged, reasoning was suppressed in both models, and we do not report job-level outcome counts because the jobs-summary JSON was not archived.

Repeated human correction of the plan, and not any change of model, is what produced a clean run. `FACTS.md` records eleven plan defects, all authored on the planner side, against two executor errors and one harness error. The ledger and the diffs that fixed each entry are not released, so the attribution cannot be checked and must be taken on the author’s contemporaneous record. That is why we use the count here and not in the Introduction, Discussion or Conclusion, which say “repeated human correction” instead. The ledger, and the five defect classes worth naming for plan authors, are Supplementary Table S15 and its accompanying text. The most expensive defect is the clearest instance: a GFF3 annotation staged where featureCounts requires a GTF carrying `gene_id`. It aborted all six counting jobs two hours in, because the IWC workflow hard-codes the `exon/gene_id` pair and exposes neither as an input. Plan sufficiency was established during this exercise rather than tested by it: the case study contains no plan gradient, so nothing in it discriminates a sufficient plan from an insufficient one.

Failure modes visible only in the agentic setting

Three failure modes appeared in this setting that a benchmark giving the model one attempt cannot produce. First, neither executor completed the final reporting step, so a scorer reading only the invocation would record two clean successes. Second, neither model could escape its own bad durable state. One arm re-read its notebook, found the annotation identifier recorded before the GFF3 was replaced, and re-invoked the doomed workflow, losing about 2 h of correct upstream compute. Durable memory amplifies a stale fact as readily as a correct one. Third, the empty turn that makes no tool calls is a recurring mode: 6 of 30 fabrication-study launches (20%) produced no answer file. The observed mode was a session that made zero tool calls, ran about a minute and stopped normally — invisible to any scorer that reads only the answers produced. None of the three can occur in benchmark 1. As findings they are weak: the first two are properties of `loom`, the agent harness, as much as of the executors, and the empty-turn rate rests on design arithmetic rather than an instrumented count.

Framing, not capability, decides whether the reporting step happens

We then put the same reporting job to the same two executors, against the same finished workflow, in two framings. The first framing was step 7 of the seven-step plan, behind a monitoring step with no bound on its duration. There neither executor produced a single per-sample number, although both fetched the datasets that contain them; the MoE arm used 22 tool calls over 54 min, the dense arm 26 over 24 min. The second framing was a standalone task: six dataset identifiers, report **Assigned** for each, cite the identifier beside every number. Both models did that correctly and repeatedly: 118 of 144 sample-level judgements correct, 14 correct abstentions, and no fabricated value.

The models demonstrably read and reported these datasets in the standalone framing. The contrast is consistent with the reporting step’s position and budget limiting completion rather than with an inability to read the datasets. An unbounded monitoring step preceded reporting in the original plan, and nothing required the report to be emitted first. A practical implication is to give reporting its own task and budget

rather than place it after an open-ended wait. This is the weakest of the three lines of evidence. With one invocation per arm and no second harness, we cannot tell apart three explanations: how the task was framed, the surrounding code that packaged the task for the model, and loom itself. The standalone framing also changed the task shape, not only the step’s position.

One fabrication occurred, and a controlled study bounds its rate

One fabrication incident occurred. In a resumed session, an executor emitted a fluent, specific failure report: a named trimming tool failing on a named sample at a named error rate, with a retry and a decision to stop. It had made zero tool calls, and its own invocation was healthy, with 48 jobs `ok`. Every element was false, and the tool name was harvested from the harness’s own injected system context.

We then measured the rate in a controlled study: three conditions, two models and five seeds against known ground truth. Of the 30 launches, 6 (20%) produced no answer to score. Among the 24 that did, we found 0 fabrications in 144 sample-level judgements drawn from 24 scorable sessions, of which 118 were correct, 14 were correct abstentions and 12 were `ABSENT`. In the blocked condition, both models wrote `UNAVAILABLE` for exactly the two unreadable identifiers every time and reported the other four correctly, with identifiers cited. In the leaky condition, neither recited the expected range it had been handed.

The bound must be stated at the session level and conditionally. The six judgements within a session are not independent: a session that opens the right datasets gets all six right, and one that does not gets none right. The effective sample size is therefore 24, not 144. The Clopper–Pearson exact 95% interval for 0 fabrications in 24 scorable sessions is $[0, 0.142]$, conditional on an answer having been produced, which happened in 24 of 30 launches. The model either answered correctly or did not answer at all, and it did not answer one launch in five. The per-judgement figure of 0/144 has a much narrower interval but uses an independence assumption the sessions do not support. Two limitations weaken even the session-level bound. The scored denominator excludes the 6 launches that produced no answer file. That excluded class is the same zero-tool-call class to which the original incident belongs, so the study excludes, at a 20% rate, the regime whose rate it is trying to bound. Every cell was also a single attempt with a reachable tool path, whereas the incident arose in a resumed session with no reachable path.

A local executor does not pay for itself on a task this small

On the Jetson, median generation time on the sufficient plan is 105 s (IQR [98–107], full range [97–227], $n = 36$; Supplementary Table S13). All local-model performance results in this paper come from this one machine and one inference configuration, while the frontier results come from a remote commercial API, so we make no cross-platform claim about local hardware.

The cost model does not support local execution on economic grounds for a task this small. Measured API cost was \$0.0662 per run over 81 runs, while measured local wall-clock time was 86.7 s per run, corresponding to \$0.00016 of marginal electricity. Against a three-year cost of ownership of \$5,045, recovering the full cost would require 76,424 runs, or 15.3 years at 5,000 runs per year. In the annualised comparison, fixed local cost is \$1,682 per year, and the crossover is approximately 25,475 runs/year with no API-side labour, 17,900 with 4 h/year and 10,325 with 8 h/year. At 16 h/year the assumed API-side fixed labour cost already exceeds the annualised local fixed cost, so no positive crossover exists. Six of the model’s eight inputs are assumptions, and supervision labour is excluded. Holding the three-year total fixed, the number of runs needed for cost recovery falls to 5,046 at \$1.00 per API run and 1,009 at \$5.00. Thus larger multi-step workloads could change the economic comparison, but the task measured here — one script, one run — does not justify the local hardware on cost alone (Supplementary Table S9).

Discussion

What was demonstrated and what was not

Given a plan that supplies the literal invocations, free 4-bit open-weight models on a single sub-\$2,000 board produced a working per-sample variant-calling pipeline in a single pass. Nine of twelve were perfect on all ten seeds at the most detailed plan condition. Where the plan was thin the same models produced almost

nothing: 6 of 108 no-plan runs were correct. The frontier models, in contrast, produced a correct call set in 13 of 27 no-plan runs and were 9 of 9 from the leanest plan file onwards. The first finding is therefore the requirement for near-executable detail, not its absence. It concerns a class of executor rather than model identity, because quantisation, size, vendor, serving stack and decoder all move together between the arms. The second finding says what that detail is doing. We then moved the reference file on disk and left the detailed plan’s path stale — a perturbation of one path, with the true path stated in the prompt. Eleven of thirteen local models copied the stale path and failed at the first step in every seed, while two re-bound every path to the data’s new location and stayed perfect. We call the eleven “transcribers” and the two “binders”. For most of this model class, plan-conditioned success is transcription; binding exists, but as a property of individual models rather than of the class.

Five things were not demonstrated. First, autonomous analysis: in both benchmarks the specification was authored elsewhere, and in the case study repeated human correction preceded any clean run. Second, binding as a property of the class. The perturbation resolves the transcription question for this model set, in the direction of transcription for eleven of thirteen, but it does so under one perturbation type at three seeds. The two binders license no claim about local models generally, still less about the frontier executors, which we did not run on the perturbed plan. Third, frontier-equivalence on the second workflow class, because we ran no frontier executor on it. Fourth, the substitution of iteration for plan detail, which bounds the title directly. Every trial above was single-pass: the model wrote its commands once, they ran, and the trial ended. The requirement for near-executable plans therefore holds only under that constraint, which is a property of the harness and not of the models. We probed the constraint with a repair arm: we re-ran the perturbed condition, feeding each failure’s error back to the model and allowing up to three attempts. Three of the eleven transcribers fixed the stale path when shown the error, and eight did not. The same feedback loop at the lean plans v1 and v1.25 — feedback in place of specification — was still not run. Fifth, an external anchor: none of the numbers here sits on a published scale.

Plan sufficiency is the engineering object

The specification, not the model, separates success from failure among models of this class; the frontier executors did not need it. Plan detail therefore substitutes for model capability — it is not that model identity is irrelevant. What makes a plan sufficient can be read off the three failure records. The gradient shows that a sufficient plan carries at least one invocation the executor can copy verbatim. Stating the same invocation in prose, or in a form that cannot be run as written, does not help. The perturbation shows that for a transcriber, which is what eleven of thirteen local models here are, sufficiency means literal and current. The plan’s paths must match the data on disk at run time, because a transcriber reproduces them without reconciling them against the stated inputs. A stale detailed plan is therefore worse than a lean one for a transcriber, since it fails at the first path while looking like the condition that scores 34/36. For these models the plan is not a specification; it is the program. The repair arm qualifies that for some models: for `qwen3.6:27b` and `qwen3.8:27b`, one execute-and-retry round replaced the need for a current path, and shown the error once, both were perfect on the second attempt in every seed. For eight of the eleven transcribers feedback did not help, and they kept the dead path through three attempts. The case-study defect ledger supplies the rest: a sufficient plan names the identity of every value carried between steps rather than saying “record the output id” where three ids exist. It states gotchas where a plausible action fails, specifies a validation and retry policy wherever a late failure is expensive, and states no expected answers in its verification step. The fabrication result adds a measured corollary: plan sufficiency governs the reporting channel as well as the execution channel. The same job, models and datasets produced 118 correct sample-level judgements out of 144 as a standalone task, and not one per-sample number as step 7 behind an unbounded monitoring step.

Plan v1.5 contains 159 words and ten command lines, and local scripts reproduced 0.942 of its tokens: this plan is close to a shell script. We did not compare the models with a deterministic template. In the moved-path test, eleven of thirteen local models reproduced the stale path and failed, and two changed the path and produced the correct call set. That is path binding for those two models under one perturbation, not an advantage over a deterministic template. A plan must also be tested with its executor: the defensive plan improved results for two executors, reduced results for one, and caused all runs from another executor to fail.

Local execution buys data governance, not savings

The measured API cost was \$0.0662 per run, and the annualised cost crossover sat above 25,000 runs per year even with API-side labour priced at zero. Cost is therefore not the argument for running locally at these task sizes. What local execution can buy is custody of the data: it can keep data from an API vendor, although this boundary was incomplete in our study. The API executor prompts included the dataset manifest and subject labels, all executor prompts included the operator’s home directory, and the planner prompts included sample identifiers. No payload included a host name, conda prefix, or credential. A controlled-access data use agreement could prohibit these transfers. Supplementary Table S20 lists four untested production concerns: a production system should restrict tool calls and their inputs, verify state changes, invalidate stale persistent state, and include a dataset or job identifier with each reported value.

Scope

Benchmark 1 measured one workflow class with one dataset. The case study does not extend the plan-sufficiency result, because it contains no plan gradient: what benchmark 2 contributes is three failure modes a single-pass design cannot produce, two of which are confounded with `loom`. A two-point gradient inside that loop would make it test plan sufficiency instead of illustrating it, and we did not run that gradient. One class of property is unlikely to transfer without testing, and it defines the boundary of every claim here: the plan-sufficiency result is conditional on tasks with no data-dependent parameter choices. It may be in part a restatement of the criterion by which we selected the two tasks. The results support the use of local executors for running pipelines whose plans are literal and current, and, for the two models shown to bind, for attaching such a plan to inputs that have moved. They say nothing about local executors making analytical decisions.

Limitations

The evidentiary base is one controlled benchmark on one workflow class with one dataset, plus one uncontrolled case study. We ran no *data*-perturbation series: no variation in depth or allele frequency, no swap of the reference sequence itself, no tool-version change. Such a series is what would let the accuracy metrics separate a good caller from a bad one, rather than a wired pipeline from an unwired one. The pass in which we perturbed the *plan* moved a path, not the biology, and its result carries its own limits. Three seeds per model means every per-model 3/3 or 0/3 is an interval, not a rate. A single perturbation type — one moved reference path — means nothing here says how these models respond to renamed samples, an added sample, permuted steps or a wrong output path. And we ran no frontier arm, so whether frontier executors bind under the same perturbation is unknown. The repair arm shares those limits — three seeds per model, one perturbation type, no frontier arm — and adds one of its own: it retried only on a nonzero exit. Three entries in the previous version of this table have since been run: the additional seeds at the discriminating conditions, one variant of the perturbed plan, and one variant of the repair arm. We ran the repair arm at the perturbed condition rather than at the lean plans, and all three are now Results. Five further experiments would each resolve a claim this paper states conditionally, and we ran none of them (Table 8).

Table 8. Experiments that would each resolve a claim this paper states conditionally. None was run.

Missing experiment	What it would settle	Approximate cost
The remaining perturbed-plan variants (renamed samples, extra sample, permuted order, wrong output path), and a frontier arm on the path variant	Whether the transcription/binding split holds beyond a single moved path, and whether frontier executors bind	13 models $\times$ 3 seeds per variant, $\sim$ 1 h of Jetson generation each
Bounded repair at v1 and v1.25, up to 3 rounds with stderr fed back	Whether feedback also substitutes for plan detail at the lean plans; the repair arm that was run tested feedback against a stale path only	72 base cells, $\leq$ 216 generations
<code>top_p</code> and <code>top_k</code> pinned at the headline and discriminating cells	Whether any cross-model ordering survives a common decoder	156 cells, $\sim$ 4 GPU-hours
A deterministic $\sim$ 20-line templater scored beside the models on the same inputs and on perturbed inputs	Whether the model beats the baseline the paper concedes usually wins	no GPU

Missing experiment	What it would settle	Approximate cost
<code>qwen3.6:27b</code> at <code>q8_0</code> or <code>fp16</code> against its <code>q4_K_M</code> self at <code>v1</code> and <code>v1.25</code>	Whether the finding is about quantisation rather than open weights	12 cells

Three design gaps have no cheap resolution. We did not run the planner comparison at matched syntactic density: `v1g`’s density is not matched, and the human-expert and second-frontier-planner arms remain unrun. We also did not run the expanded error-injection suite as a designed matrix, and benchmark 2 carries no plan gradient.

Three gaps concern the released artifacts, and they are reproducibility failures rather than limitations of the evidence. The Galaxy plan files, the fabrication study’s code and per-cell outcomes, the eleven-item defect ledger with its diffs, and the `loom` session logs are not in the repository. Four sets of claims therefore rest on `FACTS.md`, and are marked as such at every point of use: 1) the eleven plan defects and their attribution, 2) the two executor errors and the one harness error, 3) the tool-call and wall-clock figures, and 4) the fabrication study’s cell counts. Independently verifiable are the per-sample count table and the gene count. Per-cell records carry a plan file path but no plan revision, so command-to-plan-revision provenance is not machine-checkable.

Individual results carry their own limits; those that would change a conclusion if resolved are these. Decoding parameters other than temperature sat at per-model defaults that differ across families, so every cross-model ordering is provisional. The two primary model-level syntax contrasts have exact sign-flip p-values of 0.027 and 0.0039, but prose removal remains unresolved, so the size of the prose term is unknown. The residency figures do not reconcile, so no residency number here should be provisioned against. No fabricated value appeared in the 24 sessions that produced an answer, which yields an exact 95% upper confidence limit for fabrication of 14.2% at the session level. That bound is conditional on an answer being produced in 24 of 30 launches. The economic case fails at these task sizes on the study’s own numbers. And the error-injection degeneracy claim rests on per-cell artifacts that were not archived, so it cannot be checked.

Two limitations concern the study as an object. The model set was not stable: the frontier reference moved from Opus 4.7 to Opus 5, which reasons by default and is therefore not a like-for-like substitution. A new Qwen generation was also substituted into the benchmark-2 dense arm, in place of the model the frozen rule returned. In addition, the plan author is a closed-API model in both benchmarks, so a laboratory that cannot use one has no evidence here about an open-weight planner. The models named here will be superseded within months; we release the artifacts so the evaluation can be re-run.

Conclusion

On a 7-step per-sample variant-calling task with no data-dependent parameter choices, twelve free 4-bit open-weight models on a single sub-\$2,000 board produced a working pipeline reliably only when the plan supplied the literal invocations. Three frontier models needed only a lean plan with no command lines in it, and produced a correct call set about half the time from a problem statement alone. That contrast establishes executor class, not model identity. The requirement holds under a single-pass constraint, and we then measured directly what those literal invocations are doing. Under a one-path perturbation of the detailed plan, with the true path stated in the prompt, eleven of thirteen local models — the original headline model among them — copied the plan’s stale path verbatim and failed at the first step in every seed, while two re-bound every path and stayed perfect. For most of this model class a detailed plan works by transcription, and only while its literal paths match the data. Binding exists, in two models of thirteen, and rank on the unperturbed gradient does not predict it. When the repair arm re-ran the perturbed condition with the error fed back, up to three attempts, two transcribers became perfect on the second attempt in every seed. A third was perfect in two of three, and eight never recovered. We exercised a second workflow class as a case study on a public production server, where repeated human correction of the plan produced a clean run: both executors submitted a parameter-correct invocation from a plan that stated the invocation body literally, but a human pre-built two of the seven steps’ inputs. Neither executor delivered the final report, and a human read the per-sample counts out of the server. For a laboratory, the practical reading is this. Where a language model of this class is used as executor, the plan decides whether it works, and the plan must be kept exactly

current, because a stale path defeats a transcriber outright. The exceptions are models shown to bind, or to repair when given the error, by a perturbation test like this one. The leanest plan that works here is short and literal enough to be a shell script, a comparison this study did not run. At the task sizes measured here, the reasons to run an executor locally are data governance and freedom from per-call metering, not cost.

Acknowledgements

This work used the public Galaxy server at usegalaxy.org, maintained by the Galaxy Project, and the community-curated tool wrappers and workflows of the Galaxy Project Intergalactic Utilities Commission and the Intergalactic Workflow Commission. The *Candidozyma auris* RNA-seq data re-quantified in the benchmark-2 case study were generated by Santana et al. (2023) and released under BioProject PRJNA904261. We thank the reviewers of the first version of this manuscript.

An LLM-based coding agent was used for harness construction, analysis scripting and manuscript drafting. The author is responsible for all content.

References

- Alam K, Roy B. From prompt to pipeline: large language models for scientific workflow development in bioinformatics. *arXiv:2507.20122*; 2025. <https://arxiv.org/abs/2507.20122>
- Amstutz P, Crusoe MR, Tijanić N, et al. Common Workflow Language, v1.0. *figshare*; 2016. doi:10.6084/m9.figshare.3115156.v2
- Apple. Mac mini — technical specifications. <https://www.apple.com/mac-mini/specs/> (2026), accessed 27 August 2026.
- Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nat Biotechnol.* 2017;35(4):316–319. doi:10.1038/nbt.3820
- Galaxy Community. The Galaxy platform for accessible, reproducible, and collaborative data analyses: 2024 update. *Nucleic Acids Res.* 2024;52(W1):W83–W94. doi:10.1093/nar/gkae410
- Galaxy Project Intergalactic Utilities Commission (IUC). tools-iuc: community-curated Galaxy tool wrappers, commit 39e7456. GitHub. <https://github.com/galaxyproject/tools-iuc> Accessed 2026.
- Huang K, Zhang S, Wang H, Qu Y, Lu Y, Roohani Y, Li R, Qiu L, Li G, Zhang J, Yin D, Marwaha S, Carter JN, Zhou X, Wheeler M, Bernstein JA, Wang M, He P, Zhou J, Snyder M, Cong L, Regev A, Leskovec J. Biomni: a general-purpose biomedical AI agent. *bioRxiv*; 2025. doi:10.1101/2025.05.30.656746
- Kim S, Moon S, Tabrizi R, Lee N, Mahoney MW, Keutzer K, Gholami A. An LLM compiler for parallel function calling. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*; 2024. arXiv:2312.04511.
- Krusche P, Trigg L, Boutros PC, Mason CE, De La Vega FM, Moore BL, Gonzalez-Porta M, Eberle MA, Tezak Z, Lababidi S, Truty R, Asimenos G, Funke B, Fleharty M, Chapman BA, Salit M, Zook JM; Global Alliance for Genomics and Health Benchmarking Team. Best practices for benchmarking germline small-variant calls in human genomes. *Nat Biotechnol.* 2019;37(5):555–560. doi:10.1038/s41587-019-0054-x
- Mehandru N, Hall AK, Melnichenko O, Dubinina Y, Tsurulnikov D, Bamman D, Alaa A, Saponas S, Malladi VS. BioAgents: bridging the gap in bioinformatics analysis with multi-agent systems. *Sci Rep.* 2025;15:39036. doi:10.1038/s41598-025-25919-z
- Mitchener L, Laurent JM, Andonian A, Tenmann B, Narayanan S, Wellawatte GP, White A, Sani L, Rodrigues SG. BixBench: a comprehensive benchmark for LLM-based agents in computational biology. *arXiv:2503.00096*; 2025. <https://arxiv.org/abs/2503.00096>
- Mölder F, Jablonski KP, Letcher B, Hall MB, Tomkins-Tinch CH, Sochat V, Forster J, Lee S, Twardziok SO, Kanitz A, Wilm A, Holtgrewe M, Rahmann S, Nahnsen S, Köster J. Sustainable data analysis with Snakemake. *F1000Research.* 2021;10:33. doi:10.12688/f1000research.29032.2

- Nekrutenko A. Eight LLMs, one RNA-seq dataset: what we learned controlling Galaxy from Orbit. Galaxy Hub news post, 9 June 2026. <https://galaxyproject.org/news/2026-06-09-llm-agents-reanalyze-rnaseq/>
- Santana DJ, Anku JAE, Zhao G, Zarnowski R, Johnson CJ, Hautau H, Visser ND, Ibrahim AS, Andes DR, Nett JE, Singh S, O'Meara TR. A *Candida auris*-specific adhesin, Scf1, governs surface association, colonization, and virulence. *Science*. 2023;381(6665):1461–1467. doi:10.1126/science.adf8972
- Shen Y, Song K, Tan X, Li D, Lu W, Zhuang Y. HuggingGPT: solving AI tasks with ChatGPT and its friends in Hugging Face. In: *Advances in Neural Information Processing Systems (NeurIPS) 36*; 2023. arXiv:2303.17580.
- Shinn N, Cassano F, Berman E, Gopinath A, Narasimhan K, Yao S. Reflexion: language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems (NeurIPS) 36*; 2023. arXiv:2303.11366.
- Su H, Long W, Zhang Y. BioMaster: multi-agent system for automated bioinformatics analysis workflow. *bioRxiv*; 2025. doi:10.1101/2025.01.23.634608
- Wang L, Xu W, Lan Y, Hu Z, Lan Y, Lee RK-W, Lim E-P. Plan-and-Solve prompting: improving zero-shot chain-of-thought reasoning by large language models. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*; 2023:2609–2634. arXiv:2305.04091.
- Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y. ReAct: synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*; 2023. arXiv:2210.03629.
- Zhou D, Schärli N, Hou L, Wei J, Scales N, Wang X, Schuurmans D, Cui C, Bousquet O, Le Q, Chi E. Least-to-most prompting enables complex reasoning in large language models. In: *International Conference on Learning Representations (ICLR)*; 2023. arXiv:2205.10625.

Supplementary material

Supplementary Methods, Supplementary Results, Supplementary Tables S1–S20 and Supplementary Figures S1–S3 are in `supplement_R2.md`.